

## Exempel med problemlösning.

### Funktionsbeskrivning

*Beskrivning av in- och utdata (kommunikation med användaren), innehåll och form.*

Programmet ska kunna ta emot personuppgifter om studenter, samt deras studieresultat i poäng. Det ska även kunna ta emot personuppgifter om lärare samt deras lön.

Det som ska kunna presenteras är dels en fullständig lista av de inskrivna lärarna, dels hur många lärare det finns som tjänar över ett visst valfritt belopp.

Det ska också kunna presentera en lista över alla inskrivna studenter respektive en lista av studenter som uppnått en viss studiepoäng.

Tillägg av studenter och lärare ska kunna göras vid godtyckligt tillfälle. Alla data som skrivs in ska finnas tillgängliga vid senare tillfälle (även om programmet avslutas). Programmet ska erbjuda de olika alternativa funktionerna via en meny i textläge och resultaten presenteras som en text-lista med en person per rad.

De persondata som ska kunna skrivas in är namn och födelseår.

---

### Resonemang kring de data som programmet ska hantera:

Information om en lärare:

- Personuppgifter
- Lön

Information om en student:

- Personuppgifter
- Studieresultat.

Personuppgifter (information om en person):

- Namn
- Födelseår

Vi kommer att behöva hålla reda på alla lärare respektive alla studenter i var sin lista. Informationen måste sparas till fil, respektive återladdas när programmet startas. Man kan tänka sig att antingen göra det på kommando från användaren eller automatiskt vid programstart, funktionsbeskrivningen säger inget om detta. Första gången kunden startar programmet kommer filen att saknas, vilket vi måste komma ihåg att hantera i fallet med automatisk laddning. Samma öppning finns när det gäller att spara till fil. En sak till som vi måste hantera vid automatisk laddning resp. sparning är var filen ska läggas (säckväg) respektive filnamn. Det anges inte heller om det ska vara text- eller binärfil. Ska det vara en fil eller kan det vara en ide att dela upp det i två filer.

---

### Konstruktionsbeskrivning

*Detta är ett förslag till lösning. Uttryck på svenska det som ska utföras med hjälp av de imperativa byggstenarna sekvens, selektion och iteration. Kan även vara bra att ange dataflöden. Viktigt: Tänk*

*"Top-down", dvs betrakta problemet på en övergripande nivå och bortse då från detaljer. När du beskrivit denna, gör då en ny beskrivning för varje delproblem som du lämnade kvar, och låt bli att tänka på andra problem. Fortsätt med successiv nedbrytning av problemet i mindre delar, till dess att allting är trivialt.*

### Översta nivån:

- Ladda in data från filerna
- Start:
- Presentera valmöjligheterna för användaren. Dessa är:
  1. Skriva in lärare
  2. Skriva in studenter
  3. Lista alla lärare
  4. Visa antal lärare över x kr
  5. Lista alla studenter
  6. Lista alla studenter med minst ett visst studieresultat.
  7. Avsluta programmet.
- Ta emot användarens val och utför det begärda.
- Om "Avsluta" valdes så spara alla data till filerna och avsluta annars så fortsätt vid "Start"

### Skriv in lärare

- Start:
- - Ta in personuppgifter
  - Ta in uppgifter om lön
- Om användaren vill skriva in en till gå till "start"

### Skriv in student

- Start:
- - Ta in personuppgifter
  - Ta in uppgifter om studieresultat
- Om användaren vill skriva in en till gå till "start"

### Skriv ut student

För varje student:

- - Skriv ut studenten

### Skriv ut lärare

- För varje lärare:
- - Skriv ut läraren

### Skriv ut en student

- Skriv ut personuppgifter
- - Skriv ut studieresultat

### Skriv ut en lärare

- Skriv ut personuppgifter
- Skriv ut lön

### Skriv ut antal lärare över en viss lön

- Be användaren om lönenivån, vi kallar denna x
- Sätt antal=0
- För varje lärare:
  - - Om lärarens lön överstiger x
    - - Öka antal med 1
- Skriv ut antalet

### Skriv ut alla studenter som har ett givet minsta resultat

- Be användaren om antal poäng, vi kallar detta x
- För varje student:
  - - Om studentens poäng är minst x
      - Skriv ut studenten

### Ta in personuppgifter

- Ta in namn
- Ta in födelseår

### Skriv ut personuppgifter

- Skriv ut namn
- Skriv ut födelseår

---

Här är ett förslag till C-kod som implementerar konstruktionen. För varje punkt i problemlösningen ovan finns en C-funktion. Funktionerna är i huvudsak grupperade enligt den struct-typ de opererar på. Det blir naturligt att låta varje sådan "typ" hamna i var sin C-fil.

Som vi ser har målsättningen att inte definiera structer i h-filer inte uppfyllts.

Vi kommer att granska en alternativ (bättre) lösning till samma problem på föreläsning 8, där vi kommer att klara ut det problemet, efter att ha diskuterat nackdelen med denna lösning.

Lösningen ni ser här är dock inte kass, man kan nog se en hel del kod som är sämre än så.

[Här finns koden zippad](#) (okommenterad, detta dokument innehåller kommentarer och beskrivning av lösningsgången)

---

### Personuppgifter

```
#define MAXNAME 100
typedef struct t_person {
    char name[MAXNAME];
    int born;
} t_person;

/* static för att den inte ska exporteras */
static void ReadName(FILE *f, t_person *p)
```

```
{
    /* fgets är att föredra framför fscanf då texten kan innehållande blanka */
    fgets(p->name, NAMELEN, f);
    p->name[strlen(p->name) - 1] = 0; /* plockar bort det \n som läses in */
}

void PrintPersonal(t_person *person)
{
    printf(" %s ålder: %d", person->name, person->born);
}

void SavePerson(FILE *f, t_person *p)
{
    fprintf(f, "%s\n%d\n", p->name, p->born);
}

void LoadPerson(FILE *f, t_person *p)
{
    ReadName(f, p);
    fscanf(f, "%d\n", &p->born);
}

void GetPerson(t_person *p)
{
    printf("\nSkriv personens namn: ");
    scanf(" "); /* plockar bort ev. gamla \n från stdin */
    ReadName(stdin, p); /* varför är detta bättre än att bara göra gets */
    printf("När är %s född (årtal)? ", p->name);
    scanf(" %d", &p->born);
}
```

---

## En lärare

```
typedef struct t_teacher {
    t_person p;
    int salary;
} t_teacher;

int Salary(t_teacher *teacher)
{
    return teacher->salary;
}

void SaveTeacher(FILE *f, t_teacher *teacher)
{
    SavePerson(f, &teacher->p);
    fprintf(f, "%d\n", teacher->salary);
}

void LoadTeacher(FILE *f, t_teacher *teacher)
{
    LoadPerson(f, &teacher->p);
    fscanf(f, "%d\n", &teacher->salary);
}

void GetTeacher(t_teacher *teacher)
{
    GetPerson(&teacher->p);
    printf("\nUppge lön: ");
    scanf(" %d", &teacher->salary);
}
```

```
void PrintOneTeacher(t_teacher *teacher)
{
    PrintPersonal(&teacher->p);
    printf(" Lön = %d\n", teacher->salary);
}
```

---

## Lärare

```
typedef struct t_teachers {
    t_teacher *staff;
    int n;
} t_teachers;
```

*/\* antalet lärare först, därefter lägger vi lärarna i sekvens \*/*

```
void SaveTeachers(t_teachers *teachers)
{
    FILE *f;
    int i;
    f = fopen("teachers.txt", "w");
    if (f) {
        fprintf(f, "%d\n", teachers->n);
        for (i=0; i<teachers->n; i++)
            SaveTeacher(f, teachers->staff + i);
    }
}
```

*/\* genom att antalet ligger först blir det både enkelt och naturligt att allokerar minne för listan \*/*

```
void LoadTeachers(t_teachers *teachers)
{
    FILE *f;
    int i;
    f = fopen("teachers.txt", "r");
    if (f) {
        fscanf(f, "%d\n", &teachers->n);
        teachers->staff = (t_teacher*)malloc(sizeof(t_teacher)*teachers->n);
        if (teachers->staff) {
            for (i=0; i<teachers->n; i++)
                LoadTeacher(f, teachers->staff + i);
        }
    }
}
```

*/\* static för att den inte ska exporteras \*/*

```
static int CountAllTeachersAbove(t_teachers *teachers, int level)
{
    int i, number=0;
    for (i=0; i<teachers->n; i++)
        if (Salary(teachers->staff + i) > level)
            number++;
    return number;
}
```

```
void PrintNrofTeachersAbove(t_teachers *teachers)
{
    int x;
    printf("Vilken lönenivå: ");
    scanf(" %d", &x);
    printf("%d lärare av totalt %d har en lön över %d kr/mån\n",
        CountAllTeachersAbove(teachers, x), teachers->n, x);
}
```

*/\* Låt användaren mata in valfritt antal. Vi får inte glömma att öka på antalet och att allokerar mer minne (här p*

```
void GetTeachers(t_teachers *teachers)
{
    char answer;
    do {
        teachers->n++;
        teachers->staff = (t_teacher*)realloc(teachers->staff, sizeof(t_teacher)*te
        GetTeacher(teachers->staff + teachers->n - 1);
        printf("En till?(j/n) ");
        scanf(" %c", &answer);
    } while (tolower(answer)=='j');
}

void PrintAllTeachers(t_teachers *teachers)
{
    int i;
    for (i=0; i<teachers->n; i++)
        PrintOneTeacher(teachers->staff + i);
}

void FreeTeachers(t_teachers *teachers)
{
    free(teachers->staff);
}
```

---

## En student

```
typedef struct t_student {
    t_person p;
    int points;
} t_student;

void SaveStudent(FILE *f, t_student *student)
{
    SavePerson(f, &student->p);
    fprintf(f, "%d\n", student->points);
}

void LoadStudent(FILE *f, t_student *student)
{
    LoadPerson(f, &student->p);
    fscanf(f, "%d\n", &student->points);
}

void GetStudent(t_student *student)
{
    GetPerson(&student->p);
    printf("\nUppge poäng: ");
    scanf(" %d", &student->points);
}

void PrintOneStudent(t_student *student)
{
    PrintPersonal(&student->p);
    printf(" Poäng: %d\n", student->points);
}
```

---

## Studenter

```
typedef struct t_students {
    t_student *list;
    int n;
```

```
} t_students;

void SaveStudents(t_students *students)
{
    FILE *f;
    int i;
    f = fopen("students.txt", "w");
    if (f) {
        fprintf(f, "%d\n", students->n);
        for (i=0; i<students->n; i++)
            SaveStudent(f, students->list + i);
    }
}

void LoadStudents(t_students *students)
{
    FILE *f;
    int i;
    f = fopen("students.txt", "r");
    if (f) {
        fscanf(f, "%d\n", &students->n);
        students->list = (t_student*)malloc(sizeof(t_student)*students->n);
        if (students->list) {
            for (i=0; i<students->n; i++)
                LoadStudent(f, students->list + i);
        }
    }
}

void GetStudents(t_students *students)
{
    char answer;
    do {
        students->n++;
        students->list = (t_student*)realloc(students->list, sizeof(t_student)*students->n);
        GetStudent(students->list + students->n - 1);
        printf("En till?(j/n) ");
        scanf(" %c", &answer);
    } while (tolower(answer)!='j');
}

void PrintAllStudents(t_students *students)
{
    int i;
    for (i=0; i<students->n; i++)
        PrintOneStudent(students->list + i);
}

int Points(t_student *student)
{
    return student->points;
}

void PrintStudentsAbove(t_students *students)
{
    int i,x;
    printf("Vilken poäng-nivå: ");
    scanf(" %d", &x);
    for (i=0; i<students->n; i++)
        if (Points(students->list + i) >= x)
            PrintOneStudent(students->list + i);
}

void FreeStudents(t_students *students)
```

```
{  
    free(students->list);  
}
```

---

## Huvudprogrammet med meny

```
int main (void)  
{  
    t_students students={NULL, 0};  
    t_teachers teachers={NULL, 0};  
    int val;  
    LoadStudents(&students);  
    LoadTeachers(&teachers);  
    do {  
        printf("\n-----\n");  
        printf("1 - Skriv in lärare\n");  
        printf("2 - Skriv in studenter\n");  
        printf("3 - Skriv ut alla lärare\n");  
        printf("4 - Skriv ut antal över viss lön\n");  
        printf("5 - Skriv ut alla studenter\n");  
        printf("6 - Skriv ut alla med en viss minsta poäng\n");  
        printf("7 - Avsluta\n");  
        printf("Gör ditt val: ");  
        scanf(" %d", &val);  
        switch (val) {  
            case 1:  
                GetTeachers(&teachers);  
                break;  
            case 2:  
                GetStudents(&students);  
                break;  
            case 3:  
                PrintAllTeachers(&teachers);  
                break;  
            case 4:  
                PrintNrofTeachersAbove(&teachers);  
                break;  
            case 5:  
                PrintAllStudents(&students);  
                break;  
            case 6:  
                PrintStudentsAbove(&students);  
                break;  
            case 7:  
                break;  
        }  
    } while (val!=7);  
    SaveStudents(&students);  
    SaveTeachers(&teachers);  
    FreeTeachers(&teachers);  
    FreeStudents(&students);  
    return 0;  
}
```